Technical report — Los emal!

- **Track:** Track 2B — Los emal! (understand Zurich German with Apertus)
- **Event:** Online
- **Team:** Alina Ponomareva
- **Demo:** VIDEO_LINK

1. Summary

Newcomers to Zurich typically learn Standard German, yet everyday life (shops, offices, neighbours) happens in Zurich German. Answering in Standard German is socially accepted; the actual barrier is *understanding* the dialect. Los emal! trains exactly this. In nine everyday scenes a local's lines arrive as voice messages in Zurich German, with the text hidden until the learner asks for it; the learner answers in Standard German by voice, sees whether they understood, and can open the text, a translation, the hardest dialect words and ideas for an answer. Apertus 1.5 70B plays the local in Swiss Standard German; a translator turns each line into Zurich German, and an open text-to-speech model voices it.

Five evaluations shaped the design. Whisper understands spoken Zurich German far better than Apertus' experimental audio input (WER 30.9 % vs 88.8 %, E1). Apertus cannot judge whether a sentence is Zurich German: both models score below a judge that always says "no" (E2), so the app never grades dialect. Its Zurich German is passable but mixes in Bernese and Basel forms; a short style guide raises the share of Zurich forms from 61 % to 70 % without improving chrF (E3). Separate, focused calls work where one large prompt failed: a comprehension check notices 18 of 20 misunderstandings instead of 3 (E4). Five minutes of LoRA fine-tuning of Apertus 8B on 3,615 SwissDial sentence pairs raise chrF from 58.6 (70B with the style guide) to 76.7 and the share of Zurich forms to 100 % (E5). SwissDial's non-commercial licence keeps that model out of the Apache-licensed submission: it is published separately for research, and the submitted app uses Apertus 70B with the style guide as translator.

2. Architecture

See `docs/architecture.md` for the diagram. Per learner turn:

1. **Speech recognition** — Whisper large-v3-turbo (faster-whisper on CPU/CUDA, MLX on Apple Silicon) transcribes the answer.
2. **Role-play** — Apertus 70B writes the local's next line in Swiss Standard German (JSON with `reply_de` and `scene_done`) from a prompt with the role, what the character knows (prices, documents, free appointments), the learner in the third person, the steps of the scene, the level and a short sample dialogue. Writing Standard German separates *what* the local says from *how* it sounds in dialect: when 70B wrote Zurich German itself (still available as `DIALOGUE=gsw`), half of its lines on a scripted path were near-copies of the sample dialogue (section 5, A/B).
3. **Zurich German** — a translator renders the line: Apertus 70B with the prompt and style guide of E3 (default, the submitted setup), or any OpenAI-compatible endpoint set with `GSW_BASE_URL`, such as the fine-tuned 8B of E5. Fixed rules then post-correct it: a sentence that is exactly one of 55 greetings, thanks, farewells or stock phrases is replaced by a checked Zurich version, and nine regular expressions fix slips we saw repeatedly (other dialects' *zwoi*, *viu*; invented *Haubi*, *Gäret*, *Uf Wiederhersäge*; *Häter* for "hat es"; a dropped polite *Sie*), most of them only if the source contains the trigger. The rules cover only what we observed and were not checked by a native speaker.

4. **The answer**, in parallel with 2–3: a comprehension check (Apertus) and the learner's sentence the way a local would say it (translator; shown as "In Zurich you'd hear", marked experimental). The app trains understanding and does not correct the learner's German; the Standard German check of E4 is kept for the evaluation.
5. **Speech synthesis** — Qwen3-TTS 12Hz Base (0.6B on CPU, 1.7B on GPU/Apple Silicon) clones a *synthetic* reference voice (female or male) made once with Qwen3-TTS VoiceDesign. The chat shows typing dots until the line can be heard.
6. **Help on request** — short Apertus calls prepare the English translation (from the Standard German original), 1–3 hard words with explanations and two ideas for an answer.

That makes about seven model calls per turn, five to Apertus and two to the translator, at most two at a time per endpoint; outputs are validated (hard words not in the line and answer ideas in dialect are dropped). Each scene's first line was generated once with Apertus and reviewed, and is synthesised at start-up, so a scene starts at once. A side panel shows each scene's vocabulary from a glossary that Apertus generated and we reviewed (86 entries: 22 kept, 30 corrected, 34 added). The Gradio chat offers replay, 0.75× and 0.5× speed, text and translation on request, hints, levels A2–B2 and an English or German interface. **Robustness:** retries with back-off; one user turn for backends that reject system messages (Featherless); a switch to 8B when Public AI throttles 70B.

Target architecture (mandatory)

The project is deployable **on-premise** (a), **air-gapped** (b) and in a **sovereign Swiss cloud** (c):

- Apertus and the optional translator are reached through any OpenAI-compatible endpoint (`LLM_BASE_URL`, `GSW_BASE_URL`): a self-hosted vLLM server (on-premise / air-gapped) or Swiss-hosted APIs (Public AI under Swiss jurisdiction, Infomaniak with data centres in Switzerland).
- Speech recognition and synthesis are open-weights models inside the container. The learner's audio never leaves it; only text is sent to the Apertus endpoint.

External dependencies. Build time: Python packages and the open speech weights from Hugging Face (Whisper large-v3-turbo, Qwen3-TTS 0.6B Base), baked into the image (6.6 GB). Runtime: only the Apertus endpoint (and the translator endpoint, if set), which can be local.

3. Use of Apertus

- **Models:** `swiss-ai/Apertus-v1.5-70B` (default, `LLM_NAME`) for all language tasks of the app; 8B works with the same prompts and was evaluated alongside (E2–E4). A LoRA fine-tune of 8B (E5) is a separate research release, not part of the submission.
- **How it is used:** inference with prompting (the app); fine-tuning (E5, research).
- **Where it runs:** hosted endpoints in development (Public AI, Swiss jurisdiction; Hugging Face router, provider Featherless); any OpenAI-compatible endpoint, incl. self-hosted vLLM, in deployment; the fine-tuned 8B 4-bit on a laptop (`mlx_lm.server`).
- **Prompting** (`src/losemal/prompts.py`): German prompts; a JSON schema for the role-play; a compact *Zurich style guide* for translation (Zurich vs. Bernese/Basel forms, common words, perfect tense only, formal address); single-task prompts for the check and the help. Temperature 0.4 for the role-play and answer ideas, 0 otherwise.
- **Other models and their role** (as required by the track rules): Whisper large-v3-turbo (speech recognition, MIT), Qwen3-TTS 12Hz Base and VoiceDesign (speech synthesis and the synthetic reference voices, Apache-2.0). Neither produces learning content.

4. Data

- **SwissDial** (Dogan-Schönberger et al., 2021; ETH Zürich, CC BY-NC 4.0): parallel Swiss German / Standard German sentences with recordings in eight dialects. Used for evaluation (E1–E3, E5) and as training data of the E5 research model; **not** redistributed, and no SwissDial audio, text or voice is used in the app or the demo.
- **Synthetic learner test set** (`data/learner_de_testset.jsonl`, 63 items written by the author, CDLA-Permissive-2.0): context lines from the scenes with learner answers that are correct, contain one typical error, use Swiss Standard German that must not be "corrected" (*parkieren, das Tram*), or do not fit what was said. No personal data. Six items come from an SBB counter scene that the app no longer shows (its ticket vocabulary was the hardest for the translators).
- **Reviewed content:** scene openings, glossary and phrase list were generated with Apertus or written for the app and reviewed, but not by a native speaker.
- **Voices:** the app ships synthetic voices (`data/voices/`). In the demo video the male role uses a recording of a native speaker who consented to this use; it is not in the repository and is deleted after the hackathon.

5. Evaluation

The E1–E4 scripts are in `src/eval/` (`make eval-asr`, `eval-judge`, `eval-generation`, `eval-feedback`). 8B ran on Public AI; because Public AI throttled 70B, all 70B numbers and latencies come from the Hugging Face router (provider Featherless).

E1 — Who understands spoken Zurich German? 100 random SwissDial Zurich clips; both systems give the Standard German meaning, scored against the SwissDial Standard German sentence. A correct paraphrase is also penalised, so the metrics are rough; the gap is what matters.

System	WER ↓	chrF ↑	s / clip (Apple Silicon)
Whisper large-v3-turbo (MLX)	30.9 %	78.6	0.2
Apertus 1.5 8B, audio input	88.8 %	45.7	1.7

In 10 clips Apertus did not attempt the task (it said it could not hear audio, or repeated the instruction); without them it reaches WER 78.0 % / chrF 48.7 (Whisper 31.1 % / 79.0). It catches single words and numbers and invents the rest. (A PyTorch bug on Apple MPS, `F.pad`, had corrupted the logits; we fixed it first, so this is not a platform artefact.) Decision: Whisper for speech input.

E2 — Can Apertus judge Zurich German? 50 SwissDial sentences, each as Zurich German (accept), Standard German (reject) and Bernese German (reject); yes/no answer at temperature 0.

Judged correctly	70B	8B
All 150 sentences	60 %	58 %
Zurich German accepted	46 %	44 %
Standard German rejected	72 %	72 %
Bernese German rejected	62 %	58 %

A judge that always says "no" would score 67 %; 70B even "corrected" the idiomatic «Gömmer zäme en Kafi go trinke?» in a pilot. Decision: the app never grades the learner's dialect.

E3 — Zurich German generation and the style guide. 100 SwissDial sentences, Standard German → Zurich German (temperature 0). Metrics: chrF against the SwissDial transcript; the *Zurich share* of words whose form differs between Zurich and Bernese/Basel German (*hät*/*het, nöd*/*nid, gsi*/*gsy, de*/*dr*; SwissDial references: Zurich 100 %, Bernese 37 %); Standard German leftovers per 100 words, since untranslated text also scores 100 % on the share.

Model, prompt	chrF	Zurich share	Standard German leftovers / 100 words
Untranslated input (copy baseline)	50.4	100 %	5.9
70B, baseline prompt	58.6	61 %	0.1
70B, with style guide	58.6	70 %	0.1
8B, baseline prompt	59.6	61 %	0.1
8B, with style guide	58.2	71 %	0.2

The style guide shifts word forms towards Zurich but does not improve chrF; *dr*, *het* and *ond* still appear.

E4 — Checks of the learner's answers (63 synthetic items through the app's `Session`): 19 correct answers, 4 in Swiss Standard German, 20 with one typical error, 20 misunderstandings. "Before": the role-play call also judged the answer; after: separate, focused calls.

	70B before	70B	8B before	8B
False alarms on correct German	1/35	**0/43**	0/32	**0/40**
… on Swiss Standard German	0/4	0/4	0/2	0/4
Errors detected (with the expected correction)	0/19	**13/20**	0/18	**12/19**
Misunderstandings detected	2/20	**18/20**	1/20	**17/19**
False "misunderstood" alarms	0/43	**0/43**	0/43	**0/42**
No verdict / unparseable reply	9 / 1	0 / 1	13 / 1	4 / 0

The separate checks find two thirds of the errors and almost all misunderstandings without a false alarm (inside the role-play call 70B noticed 3 of 20 misunderstandings). 8B's final run lost four answers to HTTP 400 errors. The app uses the comprehension check only.

E5 — Fine-tuning Apertus 8B as a translator. We extracted the text part of Apertus v1.5 8B from the multimodal checkpoint into a standard `ApertusForCausalLM` and trained LoRA adapters (rank 16, α 32, dropout 0.05 on attention and MLP projections; 2 epochs, learning rate 1e-4, batch 16) on 3,615 SwissDial pairs Standard German → Zurich German (100 as dev set): 4.6 min on one NVIDIA RTX PRO 6000, dev loss 2.55 → 0.75. Held-out sentences, greedy decoding, metrics of E3:

Model, prompt (the 100 E3 sentences)	chrF	Zurich share	Leftovers / 100 words
70B (API), style guide	58.6	70 %	0.1
8B (API), style guide	58.2	71 %	0.2
8B text part (local), style guide	57.5	68 %	0.1
8B fine-tuned	**76.7**	**100 %**	0.1

On all 450 held-out sentences: 58.5 / 66 % before, 75.2 / 100 % after. On 90 short sentences from the app (no native reference) 98 % of the marker words were Zurich forms (70B: 80 %), and the model added no content (70B added sentences twice). Part of the gain is SwissDial's spelling; the model keeps some Standard German words (*Tüte*) and changed the meaning of two sentences. It is published separately (`alise441/apertus-v1.5-8b-de-to-zurich`, `…-mlx-4bit`, scripts included) under CC BY-NC 4.0: the hackathon terms (§6) release submitted weights under Apache-2.0, which SwissDial's licence does not allow. We found no openly licensed alternative of comparable quality and little permissive parallel data (Tatoeba deu–gsw, about 300 pairs, CC BY; Swiss German Wikipedia pairs, CC BY-SA, encyclopedic).

A/B — who writes the dialect? Six scripted scenes (three following the sample dialogue, three leaving it), the same learner lines, 21–22 lines of the local per scheme:

- *70B writes Zurich German:* fluent, but on the scripted path 5 of 10 lines were near-copies of the sample dialogue in its prompt.

- *70B writes Standard German, 70B translates:* coherent; slips into other dialects (*Des, goh, viu, zwo*) and once a changed meaning («Das macht 7.40» → «Was macht das? 7.40.»); 15 of 17 marker words in Zurich form.
- *70B writes Standard German, fine-tuned 8B translates:* the most consistent dialect (17 of 17), but invented words for rare words, greetings and ticket terms («Uf Wiederhersäge» for *Auf Wiedersehen*, «Rueer» for *retour*, «Zitig Klass» for *zweite Klasse*).

We chose Standard German plus translation, added the phrase list and fixes, and dropped the SBB scene; the default translator is 70B, so the submission stays Apache-2.0.

Latency (Apple Silicon, 70B via Hugging Face/Featherless; medians): role-play 2.3 s (36 calls; 8.6 s for the earlier call that also returned translations and words), comprehension check 1.6 s, hard words 5.9 s; translation by 70B 1.7 s, by the fine-tuned 8B (4-bit, MLX) 0.6 s. Qwen3-TTS 1.7B on MPS runs at about real time and Whisper (MLX) needs 0.9 s per answer. End to end, measured on the screen recording of the demo (three turns), the next line could be played 8–12 s after the learner stopped speaking.

6. Value, cost and scalability

Who it is for. Newcomers who study Standard German (international staff and students, people in integration programmes) and those who support them (integration offices, language schools, employers): Los emal! trains the step courses leave out, understanding the dialect where it is spoken.

Cost per use. Only the model calls cost per use; speech runs in the container. Counted with the Apertus tokenizer on the logged calls of two complete scenes (10 turns), a turn uses about 3,400 tokens in the submitted setup: about 2,350 for the five Apertus calls (role-play about 1,000 prompt and 35 completion tokens, hard words 310 + 150, the others 250–400 each) and 1,000 for the two translations with the style guide; with a local translator, 2,350 on 70B plus 220 on the laptop. An hour of practice (about 60 turns) needs about 200,000 tokens, about CHF 0.20 at an illustrative CHF 1 per million tokens for a hosted 70B; most of it is fixed instructions that prefix caching reuses. 8B did about as well as 70B in E2–E4 and fits on one 24 GB GPU (70B: about 140 GB); the fine-tuned translator runs 4-bit (4.5 GB) on a laptop.

Scalability. Apart from the per-browser session in memory the app is stateless, so instances can be replicated behind a load balancer; model endpoints are shared (vLLM batches sessions). Speech is the per-instance cost: on CPU the 0.6B model needs 3–4.5 s per second of audio and a container up to 7.3 GB of memory, so a deployment for many learners should run speech on a GPU.

7. Limitations

- **Zurich German quality.** Understandable, not native-like. Even with the fine-tuned translator, phrase list and fixes, each full scene recorded for the demo had several noticeable slips (an invented *Lesegsetz* for *Lesegerät*, *falls mer si möchted* "if we want it" for "if you want it", *gahted* for *gönd*). The rules cover only slips we saw; no native speaker has rated the output.
- **Content.** 70B once got a total wrong (10.90 instead of 9.90 Franken).
- **Speech recognition.** Whisper normalises towards fluent Standard German and mishears mixed words: «Okay, hier» became English "ok here"; «Cumulus» became «Komodos», and the comprehension check then reported a misunderstanding. Typed answers avoid this.
- **Speech synthesis.** A German TTS model reads the dialect: prosody and some sounds stay Standard German (word-initial *Ch* as in *Charte*). A Zurich voice would need an openly licensed corpus.
- **Evaluation size.** The learner test set is synthetic (63 items); the A/B had 21–22 lines per scheme.

- **Speed and availability.** On CPU a turn in the Docker image takes 20–50 s; a shared hosted endpoint is a single point of failure, which self-hosting avoids.

8. Reproducibility

- `make run` builds and starts the Docker image (CPU) on a clean checkout; set `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`, optionally `DIALOGUE` and `GSW_*` (`.env.example`). Give Docker at least 8 GB of memory: a clean clone peaked at 6.7 GB (limit 7.65 GB; earlier runs 7.3 GB). Speech models are ready after about 25 s; the scenes' first lines are synthesised in about two minutes on CPU.
- Native run on Apple Silicon (used for the demo): `uv venv --python 3.11 .venv && uv pip install --python .venv torch torchaudio -r requirements.txt mlx-whisper`, then `ASR_BACKEND=mlx TTS_MODEL=Qwen/Qwen3-TTS-12Hz-1.7B-Base make run-local PY=.venv/bin/python`.
- Research translator (gated on Hugging Face): `mlx_lm.server --model alise441/apertus-v1.5-8b-de-to-zurich-mlx-4bit --port 8080` and `GSW_BASE_URL=http://localhost:8080/v1` (from Docker: `host.docker.internal`).
- Evaluations: `make eval-judge eval-generation eval-feedback PY=.venv/bin/python SWISSDIAL=/path/to/swissdial`; E1 with the Swiss AI transformers fork for audio input (commit `3797303`): `make venv-asr eval-asr`. E4 ran with the earlier `DIALOGUE=gsw`; its checks do not depend on it. E5 scripts: `src/eval/e5/` (without data and weights). `make test`: 24 unit tests, no models or network.
- Hardware: MacBook Pro (Apple Silicon) for the app and E1–E4; one NVIDIA RTX PRO 6000 (Nebius) for E5.

9. Next steps

- An openly licensed conversational corpus Standard German → Zurich German, written and recorded with consenting speakers, to train a translator that can ship under Apache-2.0; native-speaker rating.
- A Zurich German voice from consenting speakers (fine-tuned Qwen3-TTS); more dialects (Bernese, Basel); spaced repetition of each scene's words; more interface languages.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. The code is Apache-2.0. The research model of E5 is published separately under CC BY-NC 4.0 because of its training data and is not part of this submission.

References

- Swiss AI Initiative. *Apertus 1.5* model cards, huggingface.co/swiss-ai.
- P. Dogan-Schönberger, J. Mäder, T. Hofmann. *SwissDial: Parallel Multidialectal Corpus of Spoken Swiss German.* arXiv:2103.11401, 2021.
- E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022.
- A. Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision* (Whisper), 2022.
- Qwen Team. *Qwen3-TTS* model cards, huggingface.co/Qwen.
- M. Popović. *chrF: character n-gram F-score for automatic MT evaluation.* WMT 2015.